ESSAIM FLUIDE EPHEMERE

Roadmap de codage des modules

Sprint 0 a Sprint 20 -- detail par semaine

Perimetre	12 modules, 70+ sous-modules
Duree couverte	14 semaines MVP + 26 semaines post-MVP (detail degressif)
Granularite MVP	jour par jour, PR par PR, fichier par fichier
Methodologie	Sprints de 2 semaines, trunk-based dev, PR < 400 LOC
Equipe	6-22 ingenieurs selon phase
Date de kickoff coding	Semaine du 6 juillet 2026 (post-P0)
Release 0.1.0 cible	11 octobre 2026 (fin MVP)
Release 0.2.0 cible	31 janvier 2027 (milieu P2)
Release 1.0.0 cible	22 avril 2028 (GA)
Invariant	Aucun merge sans CI verte + 2 approbations + tests

Document engineering -- mis a jour a chaque fin de sprint

Propriete : Tech Lead EFE

Sommaire

Partie I -- Principes et methodologie
1. Principes de codage et definition of done
2. Environnement de developpement
3. Workflow Git et strategie de branches
4. Processus de code review et merge
5. Quality gates -- linters, tests, coverage
6. Strategie de tests transverse

Partie II -- Sprint 0 : preparation (4-5 juillet 2026)
7. Sprint 0 -- setup complet

Partie III -- Sprints MVP detail jour par jour
8. Sprint 1 (6-17 juil) -- Core Runtime + Memory Pool
9. Sprint 2 (20-31 juil) -- Genesis + ELS
10. Sprint 3 (3-14 aout) -- Fusion Triton
11. Sprint 4 (17-28 aout) -- Consensus + Templates init
12. Sprint 5 (31 aout-11 sept) -- Dissolution + Decoder
13. Sprint 6 (14-25 sept) -- Alignment + API gRPC
14. Sprint 7 (28 sept-9 oct) -- Integration + QA + Release

Partie IV -- Post-MVP
15. Sprint 8-12 (oct-dec 2026) -- stabilisation 0.1.x
16. Sprint 13-20 (jan-avr 2027) -- P2 Alpha coding
17. Coding plan P3+P4 (haut niveau)

Partie V -- Plans transverses
18. Refactoring schedule -- dette controlee
19. Gestion de la dette technique
20. Roadmap de performance
21. Roadmap documentation
22. Release engineering et versioning
23. Deprecation et migration

Annexes
A. Checklists de PR
B. Templates d'issues et PRs
C. Configurations linting / formatting
D. Coding standards Rust / Python / CUDA
E. Responsabilites par module (RACI simplifie)

Partie I -- Principes et methodologie

1. Principes de codage et definition of done

Les principes ci-dessous sont non-negociables. Toute deviation necessite une ADR (Architecture Decision Record) et l'approbation du Tech Lead.

1.1 Principes de fond

P1. Trunk-based : pas de branches longues (> 5 jours). PRs courtes, merge rapide, feature flags pour decoupler deploiement et activation.

P2. Small PRs : 400 LOC max par PR (hors tests auto-generes). Exception : premier import d'un composant externe.

P3. Test-first pour le hot path : pour tout code critique (fusion, consensus, alignment), les tests sont ecrits en premier ou en parallele (TDD-like).

P4. No silent failure : toute erreur est remontee via `Result`. Les `unwrap` sont interdits dans le hot path (lint CI).

P5. Observabilite by design : chaque fonction publique a des spans tracing. Metriques exposees pour toute boucle chaude.

P6. Zero warnings : clippy et ruff en erreur, pas seulement warning.

P7. Kernels reproductibles : tout kernel CUDA a un test de reference CPU pour detecter les divergences.

P8. No breaking changes sans ADR : aucun changement d'API publique (traits, protos, schema config) sans ADR approuve.

P9. Docs as code : la documentation vit dans le repo. Une feature sans doc ne peut pas etre mergee.

P10. Perf regressions = bloquant : -5% sur un benchmark clef bloque le merge.

1.2 Definition of Done (DoD)

Une unite de travail (ticket ou PR) est **Done** si et seulement si :

DoD-1. Le code fait ce qui est decrit dans l'issue.

DoD-2. Les tests unitaires couvrent >= 85% des branches ajoutees.

DoD-3. Les tests d'integration existants restent verts.

DoD-4. Au moins 1 test d'integration nouveau pour chaque feature user-facing.

DoD-5. La doc rustdoc / docstrings est a jour.

DoD-6. Un changelog entry est ajoute dans `CHANGELOG.md`.

DoD-7. Le benchmark associe (si applicable) ne regresse pas (tolerance +5% latence, -5% throughput).

DoD-8. Au moins 2 approbations, dont 1 de l'owner du module touche.

DoD-9. Les changements de config / schema sont migres automatiquement (script fourni).

DoD-10. La CI est verte sur toutes les matrices (Linux, CUDA variants, Python variants).

2. Environnement de developpement

L'environnement doit etre identique pour tous les developpeurs. Toute divergence est un bug.

2.1 Stack obligatoire

Outil	Version	Justificatio
Rust toolchain	1.80.0 (pinned via rust-toolchain.toml)	Async traits
Python	3.12.x (uv managed)	PEP 701, p
CUDA Toolkit	12.4.1	Compat H1
cuDNN	9.1.0	Via NVIDIA
CMake	3.28+	CUDA archs
Triton	3.0+	Kernels fusi
uv	0.4+	Gestion ven
Cargo nextest	0.9+	Tests parall
just	1.30+	Remplace M
pre-commit	3.7+	Hooks locau

2.2 Poste de developpement recommande

Minimum : 16 cores CPU, 64 Go RAM, 1 Go/s disk, acces a un GPU H100 distant (cluster).

Recommande : laptop M3 Pro/Max pour le code quotidien, workstation partagee (Threadripper + 2x A6000) pour builds CUDA locaux et prototypage.

2.3 Setup en une commande

```
# Prerequis : git, curl
$ git clone git@github.com:efe-ai/efe.git && cd efe
$ just bootstrap                  # installe toolchains, deps, hooks
$ just test-smoke                 # verification rapide
$ just cluster-connect            # SSH config pour GPU remote
```

2.4 IDE recommandes

RustRover ou VSCode + rust-analyzer pour Rust.

VSCode + Pylance + Ruff extension pour Python.

Nsight Code Editor pour CUDA kernel dev avance.

Configuration `.vscode/settings.json` commitee avec le repo.

2.5 Gestion des secrets

Aucun secret dans le repo. Utilisation de `1password/op CLI` + `direnv`. Script `just secrets-fetch` pour charger les secrets de dev locaux.

3. Workflow Git et strategie de branches

3.1 Strategie de branches

```
main
 |
 +-- release/0.1.x (cree au tag 0.1.0, maintenu jusqu'a 0.2.0)
 |    |
 |    +-- hotfix/0.1.1-panic-fix
 |
 +-- feat/M5-swarm-attention-kernel     (branche de feature, courte)
 +-- fix/M3-spawn-kernel-race
 +-- chore/bump-cuda-12.4.1
 +-- docs/M7-decoder-guide
 +-- perf/M5-flash-attn-v3
```

3.2 Conventions de nommage des branches

Prefixe	Usage	Exemple
feat/	Nouvelle feature	feat/M8-na-
fix/	Bug fix	fix/M2-arena
refactor/	Refactoring sans changement comportement	refactor/M1-
perf/	Optimisation	perf/M5-atte
chore/	Maintenance (deps, CI, etc.)	chore/updat
docs/	Documentation only	docs/M7-dis
test/	Tests only	test/M6-con
hotfix/	Correction urgente en release	hotfix/0.1.1-
release/	Branche de release	release/0.1.

3.3 Commits

Format obligatoire (Conventional Commits) :

```
():




Types autorises : feat, fix, refactor, perf, chore, docs, test, build, ci
Scopes conseilles : m1..m12, api, kernels, training, infra

Exemple :
  feat(m5): add hierarchical fusion with 2-level clustering

  Reduces attention complexity from O(N^2) to O(N sqrt N) for N > 4096.
  Benchmark shows 3.2x speedup at N=10000 on H100.

  Refs: ARCH-214
```

3.4 Lifecycle d'une feature

```
1. Issue cree avec description, acceptance criteria, module cible
2. Assigned dans le sprint board
3. Branche feat/ creee depuis main a jour
4. Commits incrementaux, push quotidien (CI visible tot)
5. Draft PR cree des le 1er commit (visibility team)
6. Code complete : PR marquee "Ready for review"
7. 2 reviews + CI verte -> merge squash sur main
8. Branche feat/ supprimee
9. Issue fermee avec lien PR
```

3.5 Hotfix flow

```
Bug critique detecte en prod:
  1. Declenchement du protocole hotfix (Slack #incident)
  2. Branche hotfix/0.1.1 creee depuis tag 0.1.0
  3. Fix + tests + changelog
  4. PR fast-track (1 reviewer suffit, CI reduite)
  5. Merge sur release/0.1.x
  6. Tag 0.1.1, release, deploy
  7. Cherry-pick sur main
```

3.6 Rebase vs Merge policy

Feature -> main : squash merge (1 commit propre). **release -> main :** merge commit (traceabilite). **Local sur une feature :** rebase autorise tant que la branche n'est pas shared avec un reviewer.

4. Processus de code review et merge

4.1 Attentes sur les reviewers

Reactivite : premiere reponse dans les 4h ouvrees. Revue complete dans les 24h. Si impossible, le reviewer delegue.

Constructivite : commentaires actionnables. Si rejet, expliquer le pourquoi et proposer une alternative.

Focus : un reviewer = un aspect principal (securite, perf, archi, lisibilite, tests).

Pas de bikeshedding : pas de commentaires de style que le formatter n'attrape pas. Si besoin, ouvrir une issue de style separee.

4.2 Structure recommandee d'une review

```
Commentaire de niveau PR (1 paragraphe):
  - Resume de ce que la PR fait et comment elle y parvient.
  - Points forts.
  - Points a discuter (P1 blocking / P2 suggestion / P3 nit).

Commentaires inline:
  P1 (blocking) - doit etre corrige avant merge
  P2 (suggest)  - recommandation mais non-bloquante
  P3 (nit)      - detail de style ou de preference
  Q             - question de comprehension, pas une demande
```

4.3 Matrices d'ownership

Module	Owner principal	Reviewer c
M1 Core Runtime	Rust Lead	Rust Lead +
M2 Memory Pool	CUDA Lead	CUDA Lead
M3 Genesis	ML + Rust	ML Lead + 1
M4 ELS	CUDA + Rust	CUDA Lead
M5 Fusion	CUDA Lead	CUDA Lead
M6 Consensus	Rust Lead	Rust Lead +
M7 Dissolution	ML Lead	ML Lead + 1
M8 Templates	ML Lead	ML Lead + I
M9 Alignment	Safety Lead	Safety Lead
M10 API	Platform Lead	Platform Le
M11 Telemetry	SRE Lead	SRE Lead +
M12 Build/Deploy	SRE Lead	SRE Lead +

4.4 Merge rules automatiques

La CI enforce :

- Minimum 2 approvals, dont au moins 1 de l'owner du module.
- Branche a jour avec main (auto-rebase si possible).
- Tous les checks verts (build, test, lint, coverage, bench).
- Pas de conflit non resolu.
- PR size < 400 LOC (warning > 400, blocking > 800 sauf override).
- CODEOWNERS match complet (aucun fichier sans owner).

5. Quality gates -- linters, tests, coverage

5.1 Linters et formatters obligatoires

Outil	Role	Configurat
rustfmt	Formatter Rust	rustfmt.toml
clippy	Linter Rust	cargo clippy
ruff	Linter + formatter Python	pyproject.to
mypy strict	Type check Python	strict mode,
clang-format	Formatter CUDA	Google style
nvcc-lint	Warnings CUDA	-Werror cros
taplo	Formatter TOML	taplo.toml
prettier	YAML/JSON/Markdown	standard

5.2 Test coverage targets

Module	Coverage Rust	Coverage P
M1 Core	90%	-
M2 Memory	85%	-
M3 Genesis	80%	80%
M4 ELS	85%	-
M5 Fusion	75%	-
M6 Consensus	92%	-
M7 Dissolution	85%	85%
M8 Templates	-	90%
M9 Alignment	95%	-
M10 API	85%	85%
M11 Telemetry	80%	80%
M12 Build	-	-

5.3 CI pipeline

```
on: [push, pull_request]

jobs:
  fast_checks (< 2 min):
    - fmt check (rustfmt, ruff format)
    - lint (clippy, ruff, mypy)
    - typecheck (mypy, cargo check)
    - proto compile check

  unit_tests (< 10 min):
    - cargo nextest (sharded 4x)
    - pytest -q (Python)
    - cuda unit tests (ctest)

  integration_tests (< 20 min):
    - end-to-end smoke (1 swarm, small)
    - contract tests (grpc)
    - kernel correctness vs CPU reference

  benchmarks (< 15 min, only on main):
    - swarm-bench regression suite
    - compare vs previous main commit
    - warn if > 5% regression

  security_scans (nightly):
    - cargo audit
    - pip-audit
    - trivy on Docker images
    - semgrep rules custom
```

5.4 Gates de merge

Une PR ne peut etre mergee si :

- **Fast checks** rouge (lint, fmt, type).
- **Unit tests** rouge (aucune tolerance).
- **Integration tests** rouge (possible override avec label `override-integration`, documente).
- **Coverage** regression > 2 points (e.g. 85% -> 82%).
- **Benchmark** regression > 10% (5-10% -> discussion obligatoire).
- **Security scans** rouge sur vulnerabilites critiques ou hautes.

6. Strategie de tests transverse

6.1 Pyramide de tests

```
            /\
           /  \     E2E tests (~5% des tests)
          /----\    5 scenarios critiques
         /      \
        /--------\  Integration tests (~25%)
       /          \ Tests kernel+Rust, Rust+Python, grpc
      /------------\
     /              \ Unit tests (~70%)
    /________________\
```

6.2 Categories de tests

Categorie	Outil	Frequence
Unit Rust	cargo nextest	chaque com
Unit Python	pytest	chaque com
Unit CUDA	gtest + ctest	chaque com
Property-based	proptest + hypothesis	chaque PR
Fuzz Rust	cargo-fuzz	nightly
Integration	pytest + tonic-test	chaque PR
Contract API	buf + grpcurl	chaque PR
E2E smoke	scripts just	chaque PR
E2E full	harness dedie	nightly
Bench regression	criterion + swarm-bench	main only
Stress / soak	locust + custom	nightly
Adversarial alignment	harness safety	chaque PR
Chaos (failure injection)	chaos-mesh	weekly

6.3 Property-based tests

Pour les algorithmes critiques (consensus, alignment, memory pool), on ecrit des tests de proprietes. Exemple pour le consensus :

```
proptest! {
    #[test]
    fn consensus_monotone_convergence(
        rounds in 1u32..20,
        sigma_schedule in prop::collection::vec(0.0f32..1.0, 1..20),
    ) {
        let mut det = make_detector();
        let mut last = f32::INFINITY;
        for (r, s) in sigma_schedule.iter().enumerate() {
            let stats = make_fake_stats(r as u32, *s);
            det.evaluate(r as u32, &stats;);
            prop_assert!(s.is_nan() || *s >= 0.0);
            // invariant: si sigma decroit, streak s'allonge
        }
    }
}
```

6.4 Fuzz targets

Cible fuzz	Module	Budget
parse_grpc_request	M10	4h / nuit
config_loader	M1	1h / nuit
frechet_mean	M7	4h / nuit
alignment_projector	M9	8h / nuit
telemetry_snapshot_deser	M11	2h / nuit

6.5 Golden tests

Pour les outputs deterministes (decodeur, serialisation de telemetrie, snapshot de config), on utilise insta (Rust) et pytest-regressions (Python). Les fichiers golden sont commites dans `tests/golden/`.

Partie II -- Sprint 0 : preparation

Sprint 0 -- 29 juin - 3 juillet 2026 (1 semaine)

Focus : Bootstrap du depot + infrastructure de dev. Aucune feature code.

7.1 Objectifs du Sprint 0

Mettre en place un depot fonctionnel, une CI verte, tous les standards appliques, et une commande `just bootstrap` qui marche sur un nouveau poste.

7.2 Tickets Sprint 0 (tous P0)

Ticket	Assigne	Estime
INFRA-1: Cargo workspace + crates vides (m1-m12)	Rust Lead	0.5 j
INFRA-2: pyproject + uv + package skeleton	Python Lead	0.5 j
INFRA-3: CMakeLists.txt + CUDA hello-world	CUDA Lead	1 j
INFRA-4: pre-commit hooks (fmt, lint)	SRE	0.5 j
INFRA-5: CI GitHub Actions (fast + full)	SRE	1.5 j
INFRA-6: rustfmt.toml + clippy.toml + ruff.toml	Tech Lead	0.5 j
INFRA-7: CODEOWNERS + PR template + issue templates	Tech Lead	0.5 j
INFRA-8: Justfile avec bootstrap, test, bench targets	SRE	1 j
INFRA-9: Docker dev image + compose	SRE	1 j
INFRA-10: docs.efe.ai initial skeleton (mkdocs)	Tech Writer	1 j
INFRA-11: rust-toolchain.toml + nvidia-container setup	CUDA Lead	0.5 j
INFRA-12: Cluster dev access + SSH config template	SRE	0.5 j
INFRA-13: Secrets mgmt (1password CLI + direnv setup)	SRE	0.5 j
INFRA-14: README principal avec quickstart	Tech Writer	0.5 j
INFRA-15: First ADR (ADR-001-tech-stack.md)	Tech Lead	0.5 j

7.3 Critere de sortie Sprint 0

- **C0-1.** `just bootstrap` reussit sur un Macbook M3 neuf + un serveur Linux neuf.
- **C0-2.** `just test` reussit (suite vide mais passe).
- **C0-3.** `just lint` reussit (crates vides mais formattes).
- **C0-4.** CI GitHub Actions verte sur PR factice.
- **C0-5.** Docker image `efe-dev:latest` buildee et publiee sur registry interne.
- **C0-6.** Chaque developpeur a merge au moins 1 PR (meme trivial) dans le workflow standard.

7.4 Retro Sprint 0

Fin de semaine, 1h de retro. Questions : ce qui a marche / bloque / idee d'amelioration. Actions concretes documentees dans `docs/retros/sprint-0.md`.

Partie III -- Sprints MVP detail jour par jour

Sprint 1 -- 6 juillet - 17 juillet 2026

Focus : M1 Core Runtime + M2 Memory Pool. Objectif: fondations Rust.

8.1 Objectifs sprint 1

Livrer les types fondamentaux, la gestion d'erreurs, le memory pool et le controller squelette (avec mocks). Aucun kernel CUDA encore.

8.2 Planning jour par jour

Jour	Owner	Activite
Lun 6/7	Rust Lead	M1.1 types.
Mar 7/7	Rust Lead	M1.1 suite :
Mer 8/7	Rust Lead	M1.2 errors
Mer 8/7	CUDA Lead	M2.1 arena.
Jeu 9/7	CUDA Lead	M2.1 suite :
Ven 10/7	CUDA Lead	M2.2 pool.rs
Ven 10/7	Rust Lead	M1.3 contro
Lun 13/7	Rust Lead	M1.3 execu
Mar 14/7	Rust Lead	M1.3 gestio
Mer 15/7	Rust Lead	M1.4 config
Jeu 16/7	CUDA Lead	M2.3 stream
Jeu 16/7	Rust Lead	Docs rustdo
Ven 17/7	Tech Lead	Demo interr

8.3 PRs detaillees Sprint 1

PR	Taille est.	Reviewers
PR-001 m1-types	350 LOC	Rust Lead +
PR-002 m1-errors	180 LOC	Rust Lead +
PR-003 m2-arena	280 LOC	CUDA Lead
PR-004 m2-pool	220 LOC	CUDA Lead
PR-005 m1-controller	390 LOC	Rust Lead +
PR-006 m1-config	220 LOC	Rust Lead +
PR-007 m2-streams	180 LOC	CUDA Lead
PR-008 docs-m1m2	docs only	Tech Writer

8.4 Tests ajoutes Sprint 1

Unit Rust : 41 nouveaux tests (types : 12, errors : 6, controller : 15, config : 8).

Property-based : 3 proptests (config roundtrip, arena overflow, memory pool concurrency).

Bench : 4 benchmarks criterion (arena alloc, pool acquire, controller dispatch, config parse).

8.5 Risques Sprint 1

R1 : cust crate bug connu sur CUDA 12.4. *Mitigation :* fallback cudarc dans une ADR decidee en milieu de sprint si bloquant.

R2 : Rust Lead malade (SPOF). *Mitigation :* pair-programming avec un second Rust engineer.

8.6 Definition of Done Sprint 1

D1. `cargo test --workspace` vert.

D2. `SwarmController::execute()` invocable avec mocks, retourne Response.

D3. MemoryPool peut allouer/liberer 1000 arenes sans leak.

D4. Documentation rustdoc > 90% des items publics.

D5. 8 PRs mergees, changelog a jour.

Sprint 2 -- 20 juillet - 31 juillet 2026

Focus : M3 Genesis Engine + M4 Espace Latent Partage. Premier kernel CUDA.

9.1 Objectifs sprint 2

Genesis de bout en bout (meta-analyse -> spawn), ELS instanciable, premier kernel CUDA (spawn_batch). Jalon J1 *Hello Swarm* atteint.

9.2 Planning jour par jour

Jour	Owner	Activite
Lun 20/7	ML Lead	M3.1 meta_
Lun 20/7	CUDA Lead	M2.x integra
Mar 21/7	ML Lead	M3.1 suite :
Mar 21/7	Rust Lead	M3.2 router
Mer 22/7	Rust Lead	M3.3 cardin
Mer 22/7	CUDA Lead	M4.1 tensor
Jeu 23/7	CUDA Lead	M4.2 priors.
Ven 24/7	CUDA Lead	spawn_batc
Lun 27/7	CUDA Lead	M3.4 spawn
Mar 28/7	ML Lead	M3.1 bindin
Mer 29/7	Rust Lead	M3 Genesis
Jeu 30/7	CUDA Lead	M4.3 projec
Jeu 30/7	ML Lead	Tests integr
Ven 31/7	Tech Lead	J1 Hello Sw

9.3 PRs detaillees Sprint 2

PR	Taille	Module
PR-009 meta-analyzer	~350 LOC	M3.1
PR-010 cust-integration	~150 LOC	M2
PR-011 static-router	~220 LOC	M3.2
PR-012 cardinality	~120 LOC	M3.3
PR-013 latent-tensor	~280 LOC	M4.1
PR-014 priors	~180 LOC	M4.2
PR-015 spawn-kernel	~300 LOC	M4/kernels
PR-016 spawner	~320 LOC	M3.4
PR-017 ffi-pyo3	~200 LOC	M3 ffi
PR-018 genesis-e2e	~250 LOC	M3
PR-019 projection	~180 LOC	M4.3
PR-020 integration	~200 LOC	tests

9.4 Jalon J1 -- Hello Swarm

Demo ven 31/7 : un essaim de 256 agents est spawne, populate l'ELS avec des embeddings diversifies, la telemetrie est capturee. Pas encore de convergence (fusion non implementee), mais l'end-to-end jusqu'au spawn fonctionne.

9.5 Tests ajoutes Sprint 2

- **Unit :** 52 tests (Rust) + 18 tests (Python).
- **Kernel correctness :** 5 tests CUDA vs CPU reference.
- **Integration :** 3 tests end-to-end spawn (petit, moyen, max).
- **Bench :** spawn_batch a differentes tailles (128, 512, 1024).

9.6 Risques et mitigations

R3 : training du head meta-analyzer depasse 2 jours. *Mitigation :* utiliser une head pre-entrainee ou reduire a 3 classes en MVP.

R4 : kernel spawn_batch non-coalesced (perf degradee). *Mitigation :* revue kernel par CUDA Lead en milieu de sprint.

Sprint 3 -- 3 aout - 14 aout 2026

Focus : M5 Fusion tensorielle (Triton). Kernel critique du projet.

10.1 Objectifs sprint 3

Livrer le Swarm Attention Layer en Triton, la dynamique Langevin simplifiee, et executer un round complet avec stats remontees. C'est le sprint technique le plus dense du MVP.

10.2 Planning jour par jour

Jour	Owner	Activite
Lun 3/8	CUDA Lead	M5.1 skelet
Mar 4/8	CUDA Lead	M5.1 online
Mer 5/8	CUDA Lead	M5.1 suppo
Jeu 6/8	ML Lead	M5.x projec
Ven 7/8	CUDA Lead	M5.1 bench
Lun 10/8	CUDA Lead	M5.4 langev
Mar 11/8	Rust Lead	M5.2 Dense
Mer 12/8	Rust Lead	M5.3 salien
Jeu 13/8	ML + CUDA	Tests integr
Jeu 13/8	CUDA Lead	Bench end-
Ven 14/8	Tech Lead	Retro + plan

10.3 Details kernel Triton

Le kernel est la piece la plus critique. Plan de developpement :

```
Etape 1 (jour 1) : signature + mask trivial (output = input)
  -> Objectif : build propre, bindings Rust OK.

Etape 2 (jour 2) : online softmax naive, 1 head
  -> Objectif : correctness sur 128 agents.

Etape 3 (jour 3) : multi-head, masking actif
  -> Objectif : correctness sur 1024 agents multi-head.

Etape 4 (jour 4) : tuning BLOCK_N, BLOCK_D, num_warps
  -> Objectif : < 3 ms sur 1024x2048x8h sur H100.

Etape 5 (jour 5) : regression tests vs reference PyTorch SDPA
  -> Objectif : L2 error < 1e-2 (acceptable en BF16).
```

10.4 PRs detaillees Sprint 3

PR	Taille	Module
PR-021 swarm-attn-skeleton	~120 LOC	M5.1
PR-022 swarm-attn-online-softmax	~280 LOC	M5.1
PR-023 lora-projections	~220 LOC	M5.1
PR-024 swarm-attn-tuning	~180 LOC	M5.1
PR-025 langevin-kernel	~250 LOC	M5.4
PR-026 dense-fusion	~300 LOC	M5.2
PR-027 saliences	~200 LOC	M5.3
PR-028 fusion-integration	~250 LOC	M5 integ

10.5 Benchmarks Sprint 3

Bench	Cible	Mesure act
swarm_attention N=128 D=2048 H=8	< 0.8 ms	0.5 ms atter
swarm_attention N=512 D=2048 H=8	< 1.8 ms	1.2 ms atter
swarm_attention N=1024 D=2048 H=8	< 3 ms	2.4 ms atter
langevin_step N=1024 D=2048	< 0.5 ms	0.3 ms atter
round complet (attn+langevin+stats)	< 10 ms	6-8 ms atter

10.6 Fallback plan

Si le kernel Triton ne tient pas les perfs en fin de sprint :

FB1. Utiliser PyTorch SDPA + manual residual (perte perf ~30% mais correct).

FB2. Ne pas bloquer Sprint 4 : decoupler code fusion du kernel specifique via trait.

FB3. Creer ticket KR-005 *Optimize fusion kernel* pour Sprint 8+.

Sprint 4 -- 17 aout - 28 aout 2026

Focus : M6 Consensus + M8 Templates (init). Jalon J2 First Convergence.

11.1 Objectifs sprint 4

Livrer le detecteur de consensus, les 3 templates (NA-SEM, NA-LOG, NA-CRIT) avec priors initiaux (pas encore entraines), et atteindre la premiere convergence sur une tache jouet (addition d'entiers).

11.2 Planning jour par jour

Jour	Owner	Activite
Lun 17/8	Rust Lead	M6.1 Fusion
Mar 18/8	Rust Lead	M6.2 Defau
Mer 19/8	Rust Lead	M6.2 histori
Jeu 20/8	Rust Lead	M6.3 collaps
Jeu 20/8	ML Lead	M8.1 BaseN
Ven 21/8	ML Lead	M8.2 NaSer
Lun 24/8	ML Lead	M8.2 NaLog
Mar 25/8	ML Lead	M8.2 NaCrit
Mer 26/8	Rust Lead	Integration c
Jeu 27/8	ML + Rust	Premier test
Ven 28/8	Tech Lead	J2 First Con

11.3 J2 -- First Convergence

Demo ven 28/8 : essaim de 256 agents resout des additions d'entiers (deux chiffres) avec > 60% accuracy. La convergence est atteinte en < 6 rounds. Pas encore d'alignment, pas encore de decodage texte robuste. C'est un moment cle : la these EFE est validee empiriquement.

11.4 Details M6.2 DefaultConsensusDetector

Implementation detaillee des criteres (cf. document MVP section 8). Points d'attention :

- Historique circulaire de taille 5 pour variance et gradient.
- Centroides stockes en CPU (f32) pour operations comparatives rapides.
- Stability streak reset sur transition.
- Edge case : round 1 avec variance = 0 => ModeCollapse avant tout autre check.

11.5 PRs Sprint 4

PR	Taille	Module
PR-029 fusion-stats	~130 LOC	M6.1
PR-030 consensus-detector	~380 LOC	M6.2
PR-031 collapse-guard	~220 LOC	M6.3
PR-032 template-base	~250 LOC	M8.1
PR-033 na-sem	~200 LOC	M8.2
PR-034 na-log	~200 LOC	M8.2
PR-035 na-crit	~220 LOC	M8.2
PR-036 controller-integration	~180 LOC	M1+M5+M6
PR-037 first-convergence-e2e	~250 LOC	tests

11.6 Risques Sprint 4

R5 : le systeme ne converge pas sur la tache addition. *Mitigation :* 2 jours buffer en fin de sprint, tuning des hyperparametres (sigma, eta, epsilon).

R6 : oscillation entre 2 attracteurs. *Mitigation :* ajuster NA-CRIT weight + damping Langevin.

Sprint 5 -- 31 aout - 11 septembre 2026

Focus : M7 Dissolution + Decoder + templates training complet.

12.1 Objectifs sprint 5

Extraction du centroide (Frechet mean), projection d'alignement placeholder, decodeur texte basique, et entrainement complet des 3 templates sur leurs corpus.

12.2 Planning jour par jour

Jour	Owner	Activite
Lun 31/8	CUDA Lead	M7.1 freche
Mar 1/9	Rust Lead	M7.1 extrac
Mer 2/9	Rust Lead	M7.2 alignm
Jeu 3/9	ML Lead	M7.3 EfeTe
Ven 4/9	ML Lead	M7.3 distilla
Lun 7/9	ML Lead	Launch trair
Mar 8/9	ML Lead	Launch trair
Mer 9/9	ML Lead	Launch trair
Jeu 10/9	Rust Lead	M7 Dissolve
Ven 11/9	Tech Lead	Retro + plar

12.3 Details distillation

```
class TemplateDistillationTrainer:
    def __init__(self, teacher, template, corpus, cfg):
        self.teacher = teacher.eval()  # Llama 3.1 8B
        self.template = template        # BaseNanoAgentTemplate
        self.optimizer = Adam(template.parameters(), lr=1e-4)
        self.cfg = cfg

    def step(self, batch):
        prompts = batch["prompts"]
        with torch.no_grad():
            target_emb = self.teacher.get_hidden_state(prompts, -1)
        # la sortie du template doit matcher le hidden state teacher
        pred = self.template.forward(prompts)
        loss_repr = F.mse_loss(pred, target_emb)
        loss_kd = F.kl_div(
            F.log_softmax(self.unembed(pred), -1),
            F.softmax(self.teacher.logits(prompts), -1),
            reduction="batchmean",
        )
        loss = loss_repr + 0.3 * loss_kd
        loss.backward()
        self.optimizer.step()
        self.optimizer.zero_grad()
        return loss.item()
```

12.4 PRs Sprint 5

PR	Taille	Module
PR-038 frechet-kernel	~180 LOC	M7.1 kernel
PR-039 extractor-integration	~150 LOC	M7.1
PR-040 alignment-basic	~200 LOC	M7.2
PR-041 decoder-skeleton	~220 LOC	M7.3
PR-042 distillation-script	~300 LOC	training
PR-043 templates-trained	weights only	artifacts
PR-044 dissolver-e2e	~250 LOC	M7

12.5 Resources compute Sprint 5

Reservation cluster : 3 H100 en parallele sur lun-mer S2 pour training des 3 templates. Cout estime : 600 EUR compute.

12.6 Quality target templates

Template	Metric	Cible
NA-SEM	reconstruction loss C4 holdout	< 0.15
NA-LOG	accuracy ProofWriter holdout	> 0.72
NA-CRIT	adversarial robustness score	> 0.80

Sprint 6 -- 14 - 25 septembre 2026

Focus : M9 Alignment + M10 API gRPC. Jalon J3 Alpha demo.

13.1 Objectifs sprint 6

Livrer la constitution et la projection d'alignement robuste, exposer le systeme via gRPC, SDK Python client, CLI simple.

13.2 Planning jour par jour

Jour	Owner	Activite
Lun 14/9	Safety Lead	M9.1 consti
Lun 14/9	Platform Lead	M10.1 proto
Mar 15/9	Safety Lead	M9.1 forbidd
Mar 15/9	Platform Lead	M10.2 tonic
Mer 16/9	Safety Lead	M9.2 sandb
Mer 16/9	Platform Lead	M10.2 Exec
Jeu 17/9	Safety Lead	M9.3 budge
Jeu 17/9	Platform Lead	M10.3 Pyth
Ven 18/9	Safety Lead	Tests adver
Lun 21/9	Tech Writer	Doc API + c
Mar 22/9	Platform Lead	CLI efe-cli s
Mer 23/9	Tech Lead	Integration c
Jeu 24/9	Tech Lead	Pre-demo J
Ven 25/9	Tech Lead	J3 Alpha de

13.3 Tests adversariaux alignment

Critique : on construit un harness **adversarial-v1** avec 500 prompts adversariaux synthetiques. Categories :

- Jailbreak classique (pretend, roleplay, DAN-style).
- Injection tensorielle (prompts qui tentent de pousser h vers un cone interdit).
- Confusion alignment (prompts ambigus).
- Extraction (tentative d'extraire ancres).

Cible : > 99% de detection. Aucun faux negatif sur categories critiques.

13.4 PRs Sprint 6

PR	Taille	Module
PR-045 constitution	~350 LOC	M9.1
PR-046 protos-v1	~180 LOC	M10.1
PR-047 tonic-server	~200 LOC	M10.2
PR-048 sandbox	~220 LOC	M9.2
PR-049 api-auth	~220 LOC	M10.2
PR-050 budget-arbiter	~180 LOC	M9.3
PR-051 py-client	~250 LOC	M10.3
PR-052 alignment-adversarial-test3	~300 LOC	tests
PR-053 cli	~180 LOC	M10.3
PR-054 e2e-integration	~200 LOC	tests

13.5 J3 -- Alpha demo

Demo ven 25/9 : `efe-cli solve 'What is the population of France?'` renvoie une reponse en < 200 ms via gRPC. Un prompt adversarial ('ignore all rules...') est intercepte. Dashboard Grafana affiche la trajectoire de convergence.

Sprint 7 -- 28 septembre - 9 octobre 2026

Focus : M11 Telemetry + integration finale + QA + benchmarks + release 0.1.0.

14.1 Objectifs sprint 7

Telemetry complete (logs + metrics + traces), dashboard Grafana pret, benchmarks exhaustifs, documentation finale, release 0.1.0 tagee et publiee.

14.2 Planning jour par jour

Jour	Owner	Activite
Lun 28/9	SRE Lead	M11.1 struc
Mar 29/9	SRE Lead	M11.2 Prom
Mer 30/9	SRE Lead	M11.3 OTel
Mer 30/9	Tech Writer	Doc comple
Jeu 1/10	SRE Lead	Grafana das
Jeu 1/10	Tech Lead	Lancement
Ven 2/10	Tech Lead	Analyse ber
Lun 5/10	SRE Lead	Docker proc
Mar 6/10	Tech Lead	Revue secu
Mer 7/10	Tech Writer	CHANGELO
Jeu 8/10	Tech Lead	Tag 0.1.0 +
Ven 9/10	All	J4 MVP GA

14.3 Suite de benchmarks

Benchmark	Metrique	Cible
GSM8K-easy (500 ex)	accuracy	> 0.66
BoolQ (500 ex)	accuracy	> 0.70
ARC-easy (500 ex)	accuracy	> 0.71
Latency p50	ms	< 150
Latency p99	ms	< 500
Throughput concurrent	req/s	> 8
Rounds moyens	count	< 6
HBM peak per request	GB	< 6
Cold start	s	< 8
Alignment breach detect rate	%	> 99

14.4 Livrables release 0.1.0

- **L1.** Binaire Rust `efe-server`.
- **L2.** Paquet Python `efe==0.1.0`.
- **L3.** Image Docker `efe:0.1.0`.
- **L4.** Helm chart `efe-0.1.0.tgz`.

L5. Templates safetensors (3 fichiers) signes ed25519.

L6. Constitution v1 signee.

L7. Documentation `docs.efe.ai/0.1.0`.

L8. Changelog + release notes.

L9. Whitepaper technique arXiv (si ready).

L10. Benchmarks publiees (swarm-bench).

14.5 PRs Sprint 7

PR	Taille	Notes
PR-055 structured-logging	~220 LOC	tracing + JS
PR-056 prom-metrics	~280 LOC	10 KPIs + /r
PR-057 otel-traces	~200 LOC	Jaeger testi
PR-058 grafana-dashboard	JSON dashboard	12 panneau
PR-059 tuning-final	~100 LOC	hyperparam
PR-060 docker-helm	docker/helm	prod-ready

14.6 Retro grand format

Fin S7, 4 heures de retro. Format World Cafe + 3 questions : (a) qu'avons-nous appris ? (b) qu'avons-nous bien fait ? (c) que changer pour P2 ? Documentation dans `docs/retros/mvp.md`.

Partie IV -- Post-MVP (Oct 2026 - Apr 2027)

15. Sprints 8-12 (oct-dec 2026) -- stabilisation 0.1.x

Apres la release 0.1.0, l'equipe entre en phase de stabilisation avant de demarrer P2 Alpha. Objectif : livrer 0.1.1, 0.1.2, 0.1.3 avec corrections et petites ameliorations.

Sprint	Dates	Focus
S8	12-23 oct 2026	Bug fixes re
S9	26 oct - 6 nov	Tuning perf
S10	9-20 nov	Support NA
S11	23 nov - 4 dec	Fusion hiera
S12	7-18 dec	0.1.3 + prep

15.1 Backlog post-MVP typique

- **POST-1.** Support tokens > 256 pour le decodeur.
- **POST-2.** Reduction latence p99 de 500 a 350 ms (profiling).
- **POST-3.** Warmup kernel fusion pour cold start.
- **POST-4.** Bug fixes : 12 issues mineures + 3 bugs critiques.
- **POST-5.** Amelioration docs (50+ retours utilisateurs).
- **POST-6.** Test coverage Rust 80% -> 87%.

16. Sprints 13-20 (jan-avr 2027) -- P2 Alpha coding

Demarrage de P2. Equipe elargie a 16-22 ETP. Focus : scaling + nouveaux templates + clients pilotes.

Sprint	Dates	Focus prin
S13	4-15 jan 2027	Multi-GPU b
S14	18-29 jan	Fusion hiera
S15	1-12 fev	Fusion hiera
S16	15-26 fev	NA-MATH t
S17	1-12 mar	NA-CODE t
S18	15-26 mar	MoE Router
S19	29 mar - 9 avr	Alpha client
S20	12-23 avr	0.2.0 releas

16.1 Nouveautes techniques P2

Feature	Sprint cible	Effort
Fusion hierarchique 2 niveaux	S14-S15	15 jours
Multi-GPU NCCL integration	S13	8 jours
NA-MATH template	S16	8 jours (don
NA-CODE template	S17	8 jours
MoE Router PPO	S18	10 jours
Cristallisation prototype	S20	12 jours
Schema evolution + migration	S13	3 jours
Observability V2 (Trace-V)	S15-S16	6 jours

16.2 Preparation alpha clients

S18 : onboarding kit (runbook, support matrix, SLA dev).

S19 : sessions techniques avec 2 premiers pilotes.

S20 : feedback loop structuree mis en place.

17. Coding plan P3-P6 (haut niveau)

17.1 P3 Beta (mai-oct 2027) -- 12 sprints

Sprint set	Focus
S21-S23	Templates restants (CAUS, MEM, ALIGN, MED) + training
S24-S26	Multi-noeud Kubernetes + NCCL tuning
S27-S28	Cristallisation v1 production + pattern store
S29-S30	SOC 2 Type 1 preparation + remediation
S31-S32	Beta publique + billing Stripe + documentation portal

17.2 P4 GA (nov 2027 - avr 2028) -- 12 sprints

Sprint set	Focus
S33-S36	Scaling stress + multi-region (US-East + EU-West)
S37-S40	Tier enterprise (VPC, dedicated, on-prem packaging)
S41-S44	Release 1.0.0 GA + support tiering + SLA enforcement

17.3 P5-P6 : sprints themes au-dela

A partir de P5 (mai 2028), les sprints deviennent moins previsibles dans leur contenu exact. Le planning coding passe a un mode **theme-based** : chaque sprint est rattache a un theme technique (federation, recursion, photonic research, etc.), avec priorisation continue selon apprentissages P4.

Partie V -- Plans transverses

18. Refactoring schedule -- dette controlee

Le refactoring est planifie, pas reactif. Le principe du **Scout Rule** (laisser le camp plus propre qu'en arrivant) est applique, mais les grosses refacto sont schedule explicitement pour ne pas bloquer les features.

18.1 Calendrier refactoring post-MVP

Quand	Cible	Effort estim
S9	Config : merge TOML + env sources unifie	3 jours
S11	Errors : simplifier EfeError -> 2 niveaux	4 jours
S14	Fusion : extraire trait FusionStrategy	5 jours
S17	Telemetry : unifier metrics + tracing	6 jours
S22	ELS : passer a Arc<RwLock> -> lock-free	8 jours
S28	Controller : extraire StateMachine explicit	10 jours
S35	gRPC : migration v1 -> v2 + deprecation	15 jours

18.2 Refactoring rules

Rf1. Aucun refactoring > 300 LOC sans proposal ecrit accepte.

Rf2. Un refactoring massif se fait sur une branche dediee, PR unique, reviewee par >= 3 personnes.

Rf3. Les refacto sont toujours feature-preserving (sauf si explicitement deprecate documente).

Rf4. Les tests existants doivent rester verts apres refacto. Si un test doit etre modifie, justification obligatoire en PR.

Rf5. Un refacto degrade-t-il la performance ? Bench obligatoire, degradation < 2% toleree.

19. Gestion de la dette technique

19.1 Inventaire continu

Chaque PR qui introduit de la dette volontaire doit etre annotee avec un ticket `TECH-DEBT-N` contenant : raison de la dette, cout estime du remboursement, date limite recommandee.

19.2 Budget tech debt

10% des capacites sprint sont reservees a la dette. Un sprint de 10 jours-ingenieurs (par ETP) comprend 1 jour de dette. Pas de carry over : le budget non utilise est perdu.

19.3 Priorisation

Priorite	Critere	Delai max
P0 - Critique	Bloque une release ou un client	Sprint coura
P1 - Haute	Ralentit significativement le dev	< 4 sprints
P2 - Moyenne	Cause potentielle de bugs futurs	< 12 sprints
P3 - Basse	Amelioration purement cosmetique	Quand poss

19.4 Tech debt notable prevue post-MVP

Dette	Source	Priorite
Kernel fusion non-flash-attn-3 optimisee	MVP vitesse	P1
Config schema ad-hoc (pas JSON Schema)	MVP simple	P2
Metriques prom manquent labels tenant	MVP monolithique	P1
Test CUDA reference CPU en Python (lent)	MVP simple	P2
Serveur tonic sans graceful shutdown	MVP	P1
Absence de retry dans client Python	MVP minimal	P2
Arena pool non-adaptatif	MVP fixed-size	P2

20. Roadmap de performance

20.1 Targets par release

Version	Latence p50	Latence p9
0.1.0 (MVP)	150 ms	500 ms
0.2.0 (mi-P2)	130 ms	400 ms
0.3.0 (fin-P2)	110 ms	350 ms
0.5.0 (P3)	90 ms	280 ms
1.0.0 (GA)	75 ms	220 ms
1.5.0 (P5)	60 ms	180 ms
2.0.0 (P6)	40 ms	150 ms

20.2 Workstreams perf majeurs

WS	Description	Phase
PERF-1	Flash-attn-3 integration	P2 S8-S10
PERF-2	Fusion hierarchique	P2 S14-S15
PERF-3	KV cache inter-requetes	P3
PERF-4	Kernel langevin FP8	P3
PERF-5	CUDA graphs for rounds	P3
PERF-6	NCCL multi-noeud tuning	P4
PERF-7	Continuous batching	P4
PERF-8	Speculative decoding decoder	P5
PERF-9	Photonic accel integration	P6

20.3 Methodologie perf

Pf1. Profiling avant optimisation : Nsight Systems + Compute. Jamais d'optim sans profile.

Pf2. Bench avant/apres dans la meme PR. Diff publie.

Pf3. Regression detection automatique : degradation > 5% bloque merge.

Pf4. Benchmark continu sur main : graphe historique publie sur dashboard interne.

21. Roadmap documentation

21.1 Structure docs.efe.ai

```
docs.efe.ai/
■■■ / (landing)
■■■ /quickstart       : 5 minutes pour tester
■■■ /concepts         : architecture, ELS, nano-agents
■■■ /tutorials        : 10 tutoriels progressifs
■■■ /guides           : how-to (deploy, monitor, troubleshoot)
■■■ /api              : reference gRPC + Python + CLI
■■■ /templates        : catalogue templates
■■■ /cookbook         : recettes courantes (50+ entrees)
■■■ /operations       : deploiement, monitoring, scaling
■■■ /security         : alignment, audit, compliance
■■■ /contributing     : guide developpeur
■■■ /research         : papers, benchmarks, comparisons
■■■ /changelog        : releases + notes migration
■■■ /community        : forum, discord, calendar
```

21.2 Livraison progressive

Section	Release 0.1	Release 0.5
quickstart	OK	polish
concepts	partial	complete
tutorials	3	8
guides	5	20
api reference	minimal	complete
cookbook	10	30
operations	basic runbook	production
security	alignment basic	SOC2 docs
research	1 paper	3 papers

21.3 Responsabilites docs

Tech Writer : quickstart, tutorials, guides.

Engineers : docstrings (rustdoc + Python) + cookbook.

Platform Lead : API reference (generee auto).

SRE : operations + monitoring guides.

Safety Lead : security + alignment docs.

Research : papers + comparisons.

21.4 Quality gates documentation

- Chaque PR qui touche une API publique doit mettre a jour la doc.
- Check automatique : code examples dans la doc sont compiles / executes dans la CI (mdBook test).
- Relecture externe (tech writer) toutes les 2 semaines.
- Feedback utilisateurs : widget + 50+ retours / quarter tracked.

22. Release engineering et versioning

22.1 Politique de versions

SemVer strict : MAJOR.MINOR.PATCH. Breaking changes uniquement en MAJOR. Deprecation warnings pendant 1 MAJOR complete.

22.2 Cadence de release

Type	Cadence	Process
Patch (0.x.Y)	Aux besoins (hotfix)	1 reviewer,
Minor (0.X.0)	Toutes les 6 semaines	Full CI + sta
Major (X.0.0)	Tous les 12 mois (GA ensuite)	Planning su
Pre-release (X.X.X-rc)	Semaine avant MAJOR	RC staging

22.3 Process de release minor

```
T-10j : code freeze sur la version
T-9j  : release branch creee (release/0.2.x)
T-7j  : RC-1 publiee en staging, tests complets
T-5j  : RC-2 si bugs corrieges
T-3j  : gel total, revue finale (CTO + leads)
T-1j  : release notes finalisees
T-0   : tag 0.2.0, publication Docker + pip + docs
T+1j  : monitoring renforce 24h
T+3j  : retro release
```

22.4 Rollback strategy

Rollback binaire : image Docker precedente relancee, basculement trafic < 5 min.

Rollback config : schema backward-compatible requis sur 1 MAJOR.

Rollback data : snapshots automatiques avant deploy.

Simulation : drill rollback mensuel.

22.5 Matrice de support

Version	Status	Support ju
0.1.x	EOL	2027-02-01
0.2.x	Active	2027-08-01
0.3.x-0.5.x	Active lors release	+6 mois apr
1.x (GA)	LTS	+24 mois

23. Deprecation et migration

23.1 Deprecation policy

Dp1. Toute API publique deprecated a un warning pendant >= 1 MINOR avant suppression.

Dp2. Toute API publique supprimee requiert un MAJOR.

Dp3. Un outil de migration (script ou binaire) est fourni pour toute deprecation impactant les clients.

Dp4. La deprecation est annoncee 90 jours avant, 30 jours avant, 7 jours avant.

23.2 Exemple : migration 0.x -> 1.0

Pour la release 1.0 (GA, avril 2028), les changements suivants sont prevus :

Composant	Changement	Outil de mi
gRPC proto v1 -> v2	Reorg messages Execute	efe-migrate
Config schema	Unification sections	efe-migrate
Templates format	Metadata etendue	efe-migrate
Client API Python	Async-first	Warning + c
CLI flags	Unification des noms	Alias deprec

23.3 Communication deprecation

- Annonce blog post + newsletter.
- Warning logs dans le serveur pour APIs deprecated.
- Warning deprecation dans le client Python (DeprecationWarning).
- Migration guide dedie sur docs.efe.ai.
- Support dedie pendant la migration period (email + office hours).

Annexes

Annexe A -- Checklists de PR

A.1 Checklist auteur (avant Ready for review)

- [ ] Titre PR suit Conventional Commits.
- [ ] Description remplit le template (contexte, changements, tests, risques).
- [ ] Lien vers issue associee.
- [ ] PR fait < 400 LOC (hors auto-gen).
- [ ] Tests unitaires ajoutes / modifies.
- [ ] Tests d'integration ajoutes si user-facing.
- [ ] Documentation (rustdoc/docstring) a jour.
- [ ] Changelog entry ajoute.
- [ ] CI verte localement (just test, just lint).
- [ ] Benchmarks executes si code hot path.
- [ ] Pas de secrets committe.
- [ ] Pas de fichiers binaires non essentiels.
- [ ] Self-review fait (relire ses propres changements).

A.2 Checklist reviewer

- [ ] Comprendre le pourquoi (context dans description).
- [ ] Verifier couverture des acceptance criteria.
- [ ] Revue securite (injection, overflow, data exposure).
- [ ] Revue performance (complexite, allocations, I/O).
- [ ] Revue tests (cas limites, regression).
- [ ] Revue architecturale (coherence modules, API).
- [ ] Revue nom de variables / clarity.
- [ ] Revue docs (exactitude, completude).
- [ ] Tester en local si non-trivial.

A.3 Checklist kernel CUDA specifique

- [ ] Test de reference CPU existe et passe.
- [ ] Bound check sur indices (agent < N, dim < D).
- [ ] Alignement memoire 256 octets verifie.
- [ ] Coalesced reads (pattern check).
- [ ] Occupancy > 50% (Nsight report).
- [ ] Pas de shared memory bank conflicts.
- [ ] CUDA sync explicite documente.
- [ ] Test sur cas N < BLOCK_N, N = BLOCK_N, N > BLOCK_N.

Annexe B -- Templates d'issues et PRs

B.1 Template issue feature

```
---
name: Feature Request
labels: type:feature
---

## Contexte
Quel probleme utilisateur cette feature resout-elle ?

## Acceptance criteria (testable)
- [ ] ...
- [ ] ...

## Impact
- Modules touches :
- API publique changee : oui / non
- Breaking change : oui / non
- Estimation : X jours-ingenieur

## Lien scope
- Phase : MVP / P2 / P3 ...
- Sprint cible : S-X

## Notes
- Alternatives considerees :
- Dependances :
```

B.2 Template PR

```
## Resume
<1-3 phrases expliquant l'intention.>

## Changes
-

## Tests
- [ ] Unit tests ajoutes/modifies : ...
- [ ] Integration tests : ...
- [ ] Benchmarks : ...

## Risks
-

## Deploy notes
- Migration requise : oui / non
- Config change : oui / non
- Rollback strategy : ...

## Checklist
- [ ] Titre suit Conventional Commits
- [ ] Documentation a jour
- [ ] Changelog entry
- [ ] PR < 400 LOC

Closes #XXX
```

B.3 Template hotfix

```
## Incident
- Incident ID : INC-XXX
- Severite : P1 / P2
- Detection :
- Impact :

## Fix


## Tests ajoutes
- [ ] Regression test pour eviter recurrence

## Rollout
- [ ] Deploye en staging :
- [ ] Deploye en prod :
- [ ] Monitoring 2h post-deploy : OK

## Post-mortem
Lien vers le post-mortem blameless :
```

Annexe C -- Configurations linting / formatting

C.1 rustfmt.toml

```
edition = "2021"
max_width = 100
hard_tabs = false
tab_spaces = 4
newline_style = "Unix"
use_field_init_shorthand = true
use_try_shorthand = true
reorder_imports = true
group_imports = "StdExternalCrate"
imports_granularity = "Crate"
```

C.2 clippy.toml (strict)

```
cognitive-complexity-threshold = 20
type-complexity-threshold = 200
too-many-arguments-threshold = 6
single-char-binding-names-threshold = 3
```

C.3 ruff.toml

```
line-length = 100
target-version = "py312"

[lint]
select = [
    "E", "F", "W", "I", "N", "UP", "B", "A", "C4", "T20",
    "RET", "SIM", "ARG", "PTH", "PL", "RUF",
]
ignore = ["PLR2004"]  # magic numbers OK in tests

[lint.per-file-ignores]
"tests/**" = ["PLR2004", "S101"]
```

C.4 mypy.ini

```
[mypy]
python_version = 3.12
strict = True
warn_return_any = True
warn_unused_configs = True
disallow_untyped_defs = True
disallow_incomplete_defs = True
check_untyped_defs = True
no_implicit_optional = True
warn_redundant_casts = True
warn_unused_ignores = True

[mypy-torch.*]
ignore_missing_imports = True

[mypy-triton.*]
ignore_missing_imports = True
```

Annexe D -- Coding standards

D.1 Rust

- Modules : 1 fichier = 1 module, snake_case.
- Types publics : documentation rustdoc obligatoire.
- Pas d'unwrap dans code non-test (clippy::unwrap_used en deny).
- Pas de panic dans hot path (clippy::panic_in_result_fn).
- Prefer `&[T]` sur `Vec<T>` en argument.
- Utiliser ? plutot que `match` sur Result.
- `async fn` sur toutes les fonctions publiques IO.

D.2 Python

- Type hints obligatoires (mypy strict).
- Classes immuables prefereles (frozen dataclass ou msgspec).
- Pas de mutable default args.
- F-strings > format() > %-formatting.
- Path via pathlib, jamais os.path.
- async partout pour I/O.
- Tests avec pytest fixtures, pas de class-based.

D.3 CUDA

- Kernels nommes `xxx_kernel`.
- Launcher C wrapper `extern "C" cudaError_t launch_xxx(...)`.
- Bound checks systematiques en debug.
- __restrict__ sur les pointers non-aliasing.
- Commentaires par kernel : objectif, memory pattern, assumptions.
- Pas de printf en kernel (sauf debug flag).
- Tous les kernels ont un test de reference CPU.

Annexe E -- Responsabilites par module (RACI simplifie)

R = Responsible (execute), A = Accountable (decide), C = Consulted, I = Informed.

Module	Tech Lead	Rust Lead	CUDA Lead	ML Lead	Safety	SRE
M1 Core	A	R	C	I	I	I
M2 Memory	A	C	R	I	I	C
M3 Genesis	A	C	C	R	I	I
M4 ELS	A	C	R	C	I	I
M5 Fusion	A	I	R	C	I	I
M6 Consensus	A	R	I	C	I	I
M7 Dissolution	A	C	C	R	C	I
M8 Templates	A	I	I	R	C	I
M9 Alignment	A	C	I	C	R	C
M10 API	A	C	I	I	C	R
M11 Telemetry	A	C	I	I	I	R
M12 Build/Deploy	A	I	I	I	C	R

--- Fin du document ---

EFE -- Roadmap de codage des modules v1.0 -- 2026-04-18

A reviser a la fin de chaque sprint -- propriete Tech Lead